PREVENCIÓN DE FRAUDE - MODELO DE MAXIMIZACIÓN DE GANANCIA

Alejandra Villalobos Camacho

1. INTRODUCCIÓN

Mercado Libre procesa miles de transacciones por segundo, y el fraude en pagos genera pérdidas económicas y deteriora la experiencia de los usuarios. El objetivo de este trabajo es construir un modelo de Machine Learning que, con los datos del sistema de prevención de fraude, estime si una transacción es fraudulenta y decida aprobarla o rechazarla de forma que se maximice la ganancia de la empresa.

La ganancia se define por una regla de negocio: cada transacción legítima aprobada deja un 25% de su monto y cada fraude aprobado pierde el 100%.

2. HIPÓTESIS

La hipótesis central es que maximizar la ganancia no equivale a maximizar una métrica de clasificación: el modelo debe estimar probabilidades confiables, y la decisión de aprobar debe derivarse de la regla de negocio.

La regla de decisión. *Para una transacción de monto m con probabilidad de fraude p, aprobarla deja una ganancia esperada positiva cuando:*

Decisión	Si es legitima	Si es Fraude
Aprobar	+0.25 x m	-m
Rechazar	0	0

Ganancia esperada de aprobar. Si la probabilidad de fraude es p, la transacción es legítima con probabilidad 1 − p:

$$Ganacia = \ 0.25 * m(1-p) - m * p$$

Rechazar siempre deja 0, así que conviene aprobar cuando la ganancia esperada es positiva:

$$0.25 * m(1-p) - m * p > 0$$

$$p < \frac{0.25}{1.25} = 0.2$$

El monto se cancela, así que el umbral es el mismo para cualquier transacción: un fraude cuesta lo mismo que el margen de cuatro ventas legítimas. Equivale a aceptar como máximo 4 pesos de compras legítimas rechazadas por cada peso de fraude evitado.

De esta hipótesis podemos decir que:

- El modelo se elige por ganancia; las demás métricas sirven para describirlo.
- El umbral de 0,2 solo funciona si las probabilidades están calibradas, así que la calibración se verifica.
- El modelo predice el riesgo; la política de negocio decide. Rechazar no significa que la transacción sea fraude, sino que su riesgo es demasiado alto para el margen que deja.

2.1. Supuestos: La hipótesis se apoya en estos supuestos:

- ✓ **La variable score es el puntaje del sistema de prevención actual.** El enunciado no lo indica, pero su nombre lo sugiere y en el análisis se comporta como un puntaje de riesgo, con más fraude a mayor valor. Por eso se usa como referencia a superar.
- ✓ **Una transacción rechazada no genera ganancia ni pérdida,** porque el producto no se despacha.
- ✓ **No se incluyen costos indirectos de rechazar a un cliente legítimo,** como su pérdida o el impacto en el vendedor. Si se incorporaran, el umbral subiría.
- ✓ **La proporción de fraude puede diferir en producción.** La tasa del dataset es exactamente 5%, lo que sugiere una muestra preparada para el ejercicio, un cambio de proporción podría requerir recalibrar el modelo.
- ✓ **Las etiquetas de fraude son correctas y están completas.** En la práctica, el fraude se confirma con semanas de madurez.
- ✓ **Todas las variables están disponibles al momento de decidir,** es decir, ninguna contiene información posterior a la transacción.

3. ANÁLISIS Y TRANSFORMACIONES DEL DATASET

El dataset contiene 150.000 transacciones entre el 8 de marzo y el 21 de abril de 2020. Cada una tiene su fecha, su monto, la etiqueta de fraude (5% de los casos), una variable llamada score y 15 variables anonimizadas, numéricas y categóricas. Se validó la base cruda, se separó por tiempo y el análisis de variables se hizo solo con entrenamiento.

Separación temporal. La tasa de fraude cambia entre semanas (de 4,3% a 5,8%), así que separar al azar mezclaría periodos y daría una evaluación optimista. Los cortes coinciden con el inicio de semana:

Conjunto	Periodo	Transacciones	Tasa de fraude
Entrenamiento	8 de marzo al 5 de abril	90.063 (60%)	5,1%
Validación	6 al 12 de abril	24.443 (16%)	5,4%
Test	13 al 21 de abril	35.494 (24%)	4,4%

3.1. **Hallazgos principales del análisis exploratorio:**

- **La pérdida se concentra en pocos fraudes caros.** El 10% de compras más caras explica el 70,6% de la pérdida, y el 0,5% de los fraudes explica el 11,8%. (*Ver figura 1)*.
- **El score actual tiene un punto ciego.** Detecta el riesgo en valores altos, pero entre 0 y 25 la tasa de fraude ronda 3% a 6%, que una regla de corte por score alto nunca atrapa. (*Ver figura 2*).
- **o es la variable más predictiva**. Con N la tasa de fraude es 22,4%; vacía, 2,1%. El vacío es estable en el tiempo (cerca de 72% todas las semanas), así que tiene significado propio.
- **El comportamiento cambia en el tiempo.** Entre el 27 de marzo y el 5 de abril el volumen diario cae a 2.000 a 2.700 transacciones, desaparece el ciclo semanal y la tasa de fraude sube a 5,4% a 5,8%. Coincide con el inicio de las cuarentenas por COVID-19 en algunos países, aunque los datos no permiten confirmarlo. Es una de las razones para separar los conjuntos por tiempo.
- **Hay un incidente de datos.** Entre el 9 y el 13 de marzo, b y c no se registraron en el 40% a 100% de las transacciones; el resto del periodo tienen cerca de 1% de nulos. Dentro de esa semana, la tasa de fraude es similar con y sin dato, así que la falla fue mayormente aleatoria.
- **La variable j.** Parece ser una categoría de producto, tiene 8.324 valores y el 71% tiene menos de 10 transacciones. El fraude se concentra en las categorías populares.
- **La madrugada concentra fraude:** de 22 a 4 horas ocurre el 8,8% de las compras y el 16,6% de los fraudes.
- **No hay fuga de información:** ninguna variable separa sospechosamente bien (AUC máximo de 0,755).
- **k se comporta como un identificador** (valores únicos, distribución uniforme, sin relación con el fraude ni con la fecha) y se excluye.

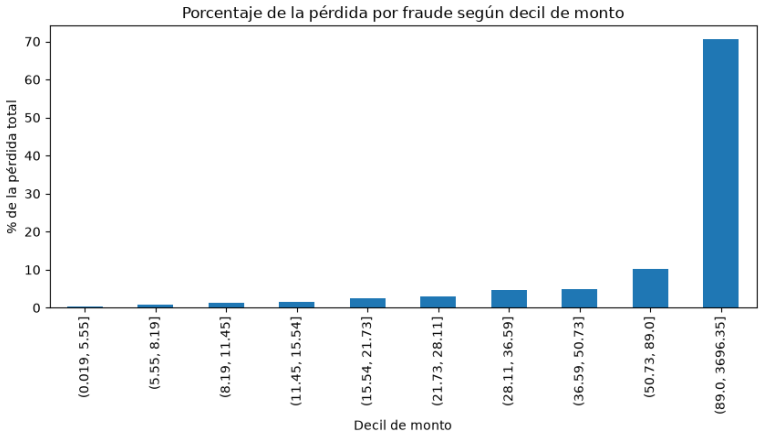
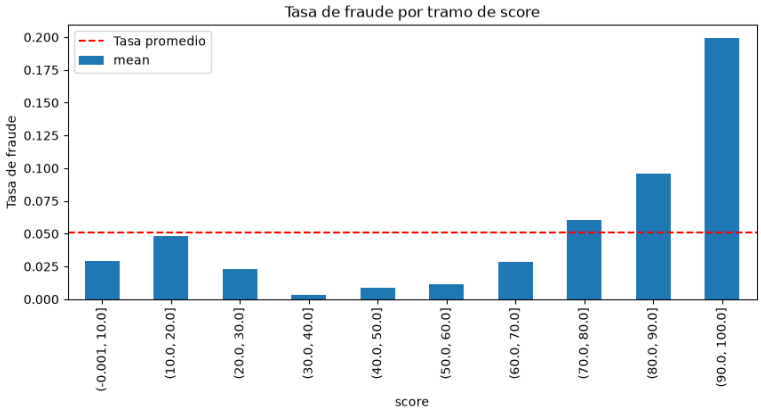


Figura 1. Porcentaje de la pérdida por fraude según el decil de monto (entrenamiento). El 10% de compras más caras explica el 70,6% de la pérdida.



Leyenda: Figura 2. Tasa de fraude según el valor del score (entrenamiento). Entre 0 y 25 la tasa ronda 3% a 6%, un fraude que una regla de corte por score alto no detecta.

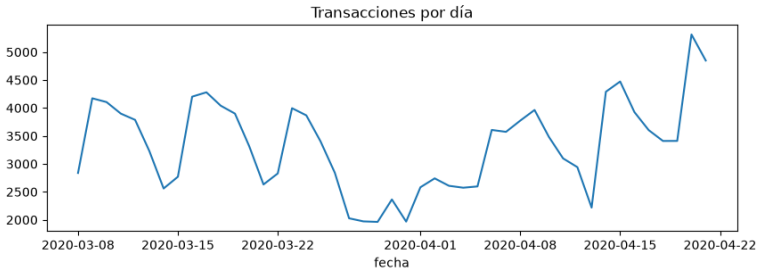


Figura 3. Transacciones por día (todo el periodo). Entre el 27 de marzo y el 5 de abril cae el volumen y desaparece el ciclo semanal, en coincidencia con el inicio de las cuarentenas por COVID-19.

3.2. **Transformaciones.** Todo lo que se aprende de los datos sale solo de entrenamiento y se aplica igual a validación, test y producción:

Variables	Tipo	Tratamiento
d, e, f, h, l, m, monto	Numéricas	Sin cambios: se conservan los valores extremos de e, los negativos de f (−1 a −5) y el tope de d en 50; los nulos de d, f, l y m quedan como faltantes
b, c	Numéricas	Sin cambios. Sus nulos, casi todos del incidente del 9 al 13 de marzo, se conservan como faltantes, sin indicador
a	Categórica (valores 1 a 4)	Se trata como categoría, no como escala: el valor 4 tiene menos fraude y los demás no siguen un orden
g	Categórica (país)	Países con menos de 100 transacciones en entrenamiento, y países nuevos, se agrupan en "otros"; nulos como "nulo"
j	Categórica (8.324 valores)	Tasa de fraude por categoría, suavizada hacia el promedio y calculada con ajuste cruzado para evitar fuga; una categoría nueva recibe la tasa promedio
o	Categórica	Tres categorías: Y, N y nulo, porque el vacío indica menos riesgo
n	Binaria	Sin cambios
p	Binaria (Y o N)	Se convierte en 1 y 0; cualquier otro valor queda nulo para no disfrazar errores
score	Numérica (0 a 100)	Solo en el escenario con score; sin cambios
fecha	Fecha y hora	Se extrae la hora del día (0 a 23) como variable; la fecha solo se usa para separar los conjuntos
k, día de la semana	—	Excluidas: k se comporta como un identificador y el día de la semana no mostró relación con el fraude

4. MODELOS UTILIZADOS

Se comparó una escalera de referencias donde cada modelo debe superar al anterior en ganancia:

1. Aprobar todo
2. El score actual
3. Una regresión logística
4. Dos modelos de boosting.

Nota: El modelo elegido es XGBoost con la codificación de `j` por tasa de fraude.

4.1. **Método de selección:**

- **Hiperparámetros:** búsqueda al azar de 30 combinaciones por algoritmo, equivalentes para LightGBM y XGBoost, medidas con log-loss en validación. Se usa log-loss y no ganancia porque la ganancia depende de unos pocos fraudes caros y es ruidosa; la log-loss es estable y premia probabilidades calibradas. Las mejores combinaciones usaron árboles simples y regularización fuerte.
- **Umbral:** 0,2 para todos los modelos. Con los hiperparámetros ajustados, el umbral que maximiza la ganancia en validación quedó entre 0,20 y 0,23, cerca del teórico.
- **Comparación:** bootstrap pareado de 1.000 simulaciones, para distinguir ventajas reales de diferencias por azar.

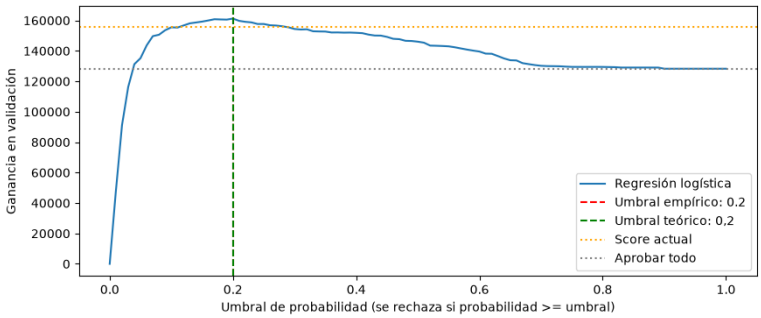


Figura 4. Ganancia en validación según el umbral de decisión. El máximo coincide con el umbral teórico de 0,2.

4.2. **Resultados en validación:**

Decisión	Resultado
Regresión logística, LightGBM y XGBoost (codificación de j por frecuencia)	Empatados entre sí: ninguna comparación superó el 70% de las simulaciones
Codificación de j por tasa de fraude suavizada	Mejoró log-loss y ganancia en ambos boosting (+2.200 a +2.500), aunque los hiperparámetros se ajustaron con la frecuencia
Algoritmo final	XGBoost: levemente superior con la tasa en log-loss, AUC y ganancia, y con experiencia previa en producción
Variante con score	Mejoró todas las métricas (log-loss de 0,154 a 0,140; AUC de 0,853 a 0,889)

4.3. **Modelos finales:** XGBoost con codificación de `j` por tasa y umbral 0,2, en dos escenarios, sin score (reemplaza al sistema actual) y con score (lo complementa). Cada uno se empaquetó con su preparación de variables, su umbral y sus metadatos antes de evaluarlo en test, para que la cifra de test corresponda exactamente al artefacto que se desplegaría.

5. Evaluación

En test, que no participó en ninguna decisión, ambos modelos superan al score actual de forma consistente. Los modelos, el umbral y el corte del score (86, elegido en validación) se aplicaron congelados.

5.1. **Ganancia con bootstrap pareado (1.000 simulaciones sobre test):**

Comparación	Gana el primero	Diferencia promedio	Rango del 90%
Sin score VS al score actual	99,3%	+10.808	3.845 a 17.896
Con score VS al score actual	100%	+19.121	13.979 a 24.101
Con score VS a sin score	99,2%	+8.314	2.612 a 13.365
Sin score VS a aprobar todo	100%	+37.906	30.065 a 46.860

En las simulaciones, incluso las menos favorables, los modelos ganan más que el score actual.

5.2. **Métricas de fraude (umbral 0,2):**

Métrica	Score actual	Sin score	Con score
Fraudes rechazados	740	646	740
Compras legítimas rechazadas	2.223	1.250	1.019
Recall en cantidad / en monto	47,1% / 49,1%	41,1% / 52,8%	47,1% / 56,8%
Precisión en cantidad / en monto	25,0% / 32,1%	34,1% / 39,2%	42,1% / 44,8%
Gini (AUC)	0,465 (0,733)	0,696 (0,848)	0,779 (0,889)
PR-AUC (al azar: 0,044)	0,229	0,344	0,444

El modelo sin score rechaza menos fraudes que el score actual, pero evita más dinero de fraude: prioriza los fraudes caros, que es lo que maximiza la ganancia. El modelo con score rechaza los mismos 740 fraudes que el score actual con menos de la mitad de compras legítimas rechazadas.

5.3. **Calibración.** Las transacciones con probabilidad entre 0,15 y 0,20 tuvieron 15,8% de fraude real (sin score) y 17,6% (con score); las de entre 0,20 y 0,30, 21,9% y 22,9%. Justo debajo del umbral la tasa real es menor a 20% y justo arriba es mayor: la regla de decisión funciona en test como indica la teoría.

5.4. **Interpretación.** Como las variables están anonimizadas, los valores SHAP se usaron para verificar la coherencia del modelo, no para explicar el fraude. Las variables más importantes (`o`, `l`, `m`, monto, `f` y `j`) y sus direcciones coinciden con el análisis exploratorio, y el modelo no aprendió el incidente de `b` como señal de riesgo.

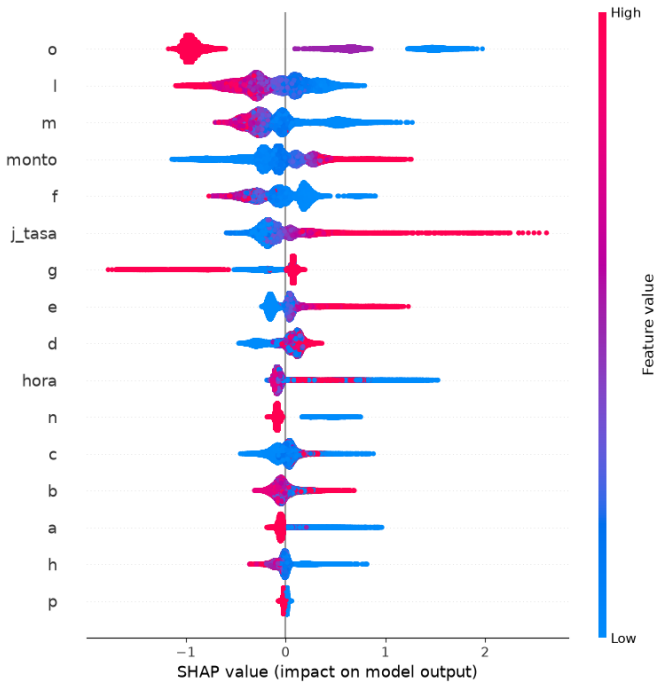


Figura 5. Importancia y dirección de las variables en el modelo sin score (test). En las variables categóricas, el color corresponde al código interno de cada categoría.

6. CONCLUSIÓN

Se recomienda el modelo XGBoost con umbral 0,2: en test gana entre 4,3% y 7,5% más que el sistema actual, evita más dinero de fraude y rechaza a casi la mitad de clientes legítimos.

- **Si el score actual seguirá funcionando,** conviene el modelo con score: es el de mayor ganancia. Su costo es depender del score, cuyos cambios podrían afectar al modelo sin aviso.
- **Si se busca reemplazar el score,** el modelo sin score también lo supera de forma consistente y no depende de él.

Las decisiones que más influyeron en el resultado fueron sobre los datos y la regla de negocio, no sobre el algoritmo: la regresión logística, LightGBM y XGBoost empataron hasta que la codificación de `j` por tasa de fraude convirtió una ventaja probable sobre el score en una consistente.

6.1. **Limitaciones:**

- Los hiperparámetros, la codificación y el modelo se eligieron en una sola semana de validación. Con solo cinco semanas de historial, la validación cruzada temporal habría entrenado sus primeras rondas con dos semanas y dado mediciones poco comparables; para compensarlo se usó bootstrap y un test intacto.
- Test es un único periodo de nueve días.
- Las variables están anonimizadas: el modelo se puede verificar, pero no explicar en términos de negocio.
- El modelo sin score sobreestima levemente el riesgo cuando baja la tasa de fraude (test tiene 4,4% frente a 5,1% de entrenamiento); conviene monitorear su calibración.

7. ¿QUÉ PASOS SEGUIR PARA QUE LA PERFORMANCE EN LABORATORIO SEA SIMILAR A LA DE PRODUCCIÓN?

La clave es evaluar el modelo como lo verá producción: siempre hacia adelante en el tiempo, con datos que no influyeron en ninguna decisión.

1. **Separar por tiempo, no al azar.** Entrenar con el pasado y evaluar con un periodo posterior. Separar al azar comparte días y campañas de fraude entre entrenamiento y evaluación, y hace que el modelo parezca mejor de lo que es.
2. **Reservar un test que no participe en ninguna decisión** y usarlo una sola vez, con el modelo seleccionado.
3. **Aprender todas las transformaciones solo de entrenamiento** y aplicarlas igual al resto. Explorar los datos también solo en entrenamiento, para que el test no influya en las decisiones.
4. **Preparar el modelo para lo que no vio:** categorías y países nuevos con un valor por defecto, y valores inesperados que queden visibles como nulos en lugar de disfrazarse de válidos.

5. **Usar en producción el mismo código de preparación que en el entrenamiento,** empaquetado junto al modelo, para evitar diferencias entre ambos.
6. **Verificar que cada variable esté disponible al momento de la decisión** en tiempo real.
7. **Considerar la madurez de las etiquetas:** el fraude se confirma con semanas de retraso, así que los periodos recientes pueden tener fraude sin reportar.
8. **Asegurar la reproducibilidad:** semilla fija, versiones fijas de las librerías y parámetros centralizados.

7.1. **Evidencia de este proyecto.** Entre validación y test, el AUC, que no depende de cuánto fraude hay, se mantuvo (0,853 a 0,848 sin score; 0,889 en ambos con score), y la tasa de aprobación también (94,7%). La mejora frente al score bajó un poco (+5,7% a +4,3% sin score): es el optimismo esperado de haber tomado decisiones en validación, y el motivo de reservar test.

8. Si la performance en producción es muy distinta a la esperada, ¿cuáles son las causas más probables?

Las causas más probables:

1. **Cambios en la calidad o el formato de los datos de entrada.** Un sistema de origen deja de enviar un campo o cambia su formato, y el modelo recibe datos distintos sin que nadie lo note. Este proyecto tiene un caso real: entre el 9 y el 13 de marzo, `b` y `c` dejaron de registrarse en hasta el 100% de las transacciones.
2. **Diferencias entre entrenamiento y producción**: una variable se calcula distinto en cada entorno.
3. **Fuga de información en el laboratorio:** el modelo usó datos futuros o variables que en producción no existen, inflando la performance esperada.
4. **Cambio en la proporción de fraude.** Si la tasa real difiere de la de entrenamiento, las probabilidades se descalibran. Ya se observa en este proyecto: con 4,4% de fraude en test frente a 5,1% en entrenamiento, el modelo sin score sobreestima levemente el riesgo.
5. **Cambio en los patrones:** la población de compras o la relación entre las variables y el fraude cambia con el tiempo. Las cuarentenas de marzo de 2020 son un ejemplo en este dataset: el volumen cayó y la tasa de fraude subió.
6. **Adaptación de los defraudadores:** el fraude es cambiante y los patrones varian en respuesta al propio modelo.
7. **Etiquetas inmaduras:** la performance medida al inicio subestima el fraude que se confirma después.
8. **Categorías nuevas:** productos, vendedores o países que el modelo no vio. Aquí se manejan con valores por defecto, pero un aumento brusco de su proporción indica que el negocio cambió.

9. ¿Qué pasos seguir para poner el nuevo modelo en producción?

El principio: nunca detener el flujo de pagos, pero tampoco aprobar a ciegas lo que el modelo no pudo evaluar.

1. **Reentrenar con todos los datos disponibles.** Usar entrenamiento y validación, con los hiperparámetros y la cantidad de árboles ya elegidos, para incorporar la semana más reciente.
2. **Validar los datos de entrada.** Un contrato de datos revisa cada campo (tipo, formato, rango) antes de llegar al modelo. Las transacciones con datos inválidos no se aprueban automáticamente: van a revisión manual o a reglas de respaldo.
3. **Correr en paralelo con el sistema actual** (*champion/challenger*). El modelo nuevo decide en modo sombra, sin afectar a los clientes, y se compara su ganancia con la del score actual una vez que maduran las etiquetas.
4. **Liberar de forma gradual:** primero a un porcentaje pequeño del tráfico, ampliando si los resultados se sostienen.
5. **Monitorear en tres niveles:**
   - **Datos, diario y sin etiquetas:** porcentaje de nulos por variable, valores fuera de rango y categorías nuevas. Una alerta de nulos habría detectado el incidente de `b` y `c` el primer día.
   - **Distribución, semanal y sin etiquetas:** PSI del score y de cada variable importante (menos de 0,1 estable; más de 0,25, cambio importante).
   - **Desempeño, con etiquetas maduras:** ganancia, Gini, calibración y tasa de aprobación.
6. **Recalibrar o reentrenar** cuando el monitoreo lo indique, y tener un plan para volver al modelo anterior si los indicadores se degradan.
7. **Revisar el umbral si cambia el negocio:** si cambia el margen o se incorpora el costo de rechazar a un cliente legítimo, se recalcula el umbral sin reentrenar el modelo.

Escala. Con miles de transacciones por segundo, el modelo debe servirse con baja latencia, calcular sus variables en tiempo real (por ejemplo, con un *feature store*) y reentrenarse sobre infraestructura distribuida. El empaquetado de este proyecto ya resuelve una parte: la preparación de variables viaja junto al modelo.

IA generativa. No se usaría en la decisión en tiempo real, por latencia y costo. Sí aporta como copiloto para los analistas en la revisión manual, resumiendo cada caso a partir de sus contribuciones SHAP, y como agente de monitoreo que detecte cambios y proponga reentrenar, siempre con aprobación humana.